SOFTWARE PROJECT GUIDE · SOURCE MAP · VIVA PREPARATION

FeedbackPro AI

A fuller technical walkthrough of the Customer Feedback Analytics demo: its pages and files, data model, analysis logic, user workflows, dependencies, limitations, and project-specific viva questions.

HTML · CSS · Vanilla JavaScript | No build step | Browser-local demo | Updated project guide

Project in one minute

FeedbackPro AI is a static browser application for collecting customer feedback, categorizing sentiment with transparent local rules, exploring actual saved records through dashboards, and managing follow-up. It provides demo sign-in, a feedback form, analytics, a rule-based insights copilot, complaints, import preview, and CSV/PDF reports.

It runs without a server-side application or package build. Records are stored in the browser's LocalStorage for the current origin and browser profile.

What "AI" means in this implementation

Sentiment uses a small word lexicon, star-rating adjustment, topic phrase lists, and urgent terms. The Copilot maps recognized question patterns to calculations over actual LocalStorage reviews and can show review excerpts. It is not a hosted LLM and does not send review text to an AI service.

Deterministic local rules · no AI secret needed

Guide contents

02 App architecture and page/script loading

03 HTML entry points and shared browser data

04 JavaScript file-by-file reference: analysis and dashboard

05 JavaScript reference: user workflows and reports

06 CSS files, data flow, formulas, and dependencies

07 Run instructions, demo walkthrough, limitations, and production path

08 Viva questions: architecture and core concepts

09 Viva questions: features, data, testing, and improvements

Scope note: This guide documents the code in the current repository. It distinguishes implemented browser-demo behavior from production SaaS capabilities that would still need a backend.

02 · ARCHITECTURE

How the application is assembled

Three HTML entry points provide separate surfaces. The dashboard is a single-page shell whose view sections are shown or hidden by JavaScript. Classic scripts share global functions, so the script include order is part of the application architecture.

1. **HTML entry page** `login.html`, `feedback.html`, or `index.html` defines markup and `data-page`.
2. **Shared services** `storage.js` initializes sample records and supplies browser-storage helpers.
3. **Feature functions** Analysis, analytics, table, charts, reports, import, complaints, and Copilot scripts build on those helpers.
4. **Render / persist** DOM views render from review records; user changes write to LocalStorage and refresh the relevant views.

Dashboard script order

`storage.js → sentiment.js → analytics.js → insights.js → reviews.js → dashboard.js → reports.js → workspace.js → app.js`

Storage provides records and utilities. Sentiment is consumed by analytics and workflows. The later feature scripts rely on these earlier global functions. `app.js` wires shared navigation and settings after feature functions exist.

Other page script order

Login: `storage.js → auth.js → app.js`.
Feedback form: `storage.js → sentiment.js → feedback.js → app.js`.

The feedback page needs sentiment analysis to preview and classify submitted text. Login does not load dashboard feature code.

Dashboard views inside `index.html`

PAGE VIEW	ROLE	PRIMARY RENDERING/INTERACTION CODE
Overview	Summary KPIs, charts, recent reviews, and insight cards	`dashboard.js`, `analytics.js`, `insights.js`
Reviews / Sentiment / Analytics / Customer insights	Record management, local classifications, deeper aggregates, theme findings	`reviews.js`, `dashboard.js`, `insights.js`
Copilot / Complaint tracking / Import data	Dataset Q&A, issue workflow, file preview and import	`workspace.js`
Reports / Notifications / Settings	Exports, alert center, theme/profile/demo data controls	`reports.js`, `dashboard.js`, `app.js`

Key design choice: A single browser-side review dataset feeds each view. Filtering and aggregation happen locally; there is no request to a remote app API.

HTML entry points and shared state

FILE	WHAT IT CONTAINS	WHY IT MATTERS / KEY CONNECTIONS
`index.html`	Main application shell: sidebar and topbar, navigation buttons, app views, dashboard/analytics containers, review modal, notifications, profile/theme controls, and report actions.	IDs and data attributes form the contract with JS: for example `dashboard-view`, `globalSearch`, `kpiGrid`, `analytics-view`, and chart/table targets. Loads Chart.js, jsPDF, SheetJS from CDNs.
`login.html`	Login/signup tabs, email/password fields, show-password controls, demo-login option, and auth UI.	Actual behavior in `auth.js`; authentication state is only local demo state.
`feedback.html`	Customer form for identity, product/category, stars, title/text, recommendation, counters, live sentiment, and submit feedback.	`feedback.js` validates inputs, classifies review, stores it, creates notification, and returns to dashboard.
`README.md`	Demo credentials, feature outline, local server command, import column aliases, dependencies, and production caveats.	Repository's quick-start and limitations reference.
`assets/images/FeedbackPro Analytics Logo.png`	Displayed brand image.	Referenced by the dashboard shell.

Browser data keys and record contents

LOCALSTORAGE KEY	CONTENTS	USED BY
`feedbackpro_reviews`	Review array and customer feedback records.	Dashboard, reviews, analysis, Copilot, import, complaints, reports.
`feedbackpro_settings`	Theme, display name, image/profile preferences, notification settings, date range.	App shell, dashboard, settings UI.
`feedbackpro_auth`	Demo flag and profile fields; not a secure session.	Login flow and dashboard redirect guard.
`feedbackpro_notifications`	Notification message, timestamp, read/unread state.	Submission and dashboard notification center.
`feedbackpro_complaints`	Issue tied to a source review, description, priority, owner, status, date.	Complaint tracking in `workspace.js`.

Shared review fields

id · customerName · email · product · category · rating · reviewTitle · reviewText · recommendation · createdAt · sentiment · sentimentScore · topics · urgent · explanation · suggestion · responseStatus · reply · replyDate.

Shared data access layer: `js/storage.js`

Defines sample records and default settings; provides getters and savers for reviews/settings/notifications; initializes demo sample data; exposes reset/auth helpers; and supplies formatting, HTML escaping, and toast utilities. `getReviews()` derives current local analysis from text/rating when records are read.

Data boundary: LocalStorage is per browser and origin. It is neither a server database nor encrypted tenant storage. Clearing site data or changing browser/profile may remove or isolate the workspace.

Analysis, dashboard, and review modules

`js/sentiment.js`

Core: `analyzeSentiment(text, rating)` . Tokenizes text; looks for positive/negative/neutral lexicon terms; checks recent tokens for negation and intensifiers; adds a bounded 1–5 star adjustment; assigns polarity by score thresholds; matches topic vocabularies; flags explicit urgent terms; returns keywords, score, explanation, and follow-up suggestion. Also summarizes sentiment counts and aggregates top positive/negative words.

`js/analytics.js`

Reusable calculations: date filtering (today/7/30/90/all/custom), average rating, rating-based satisfaction, product averages, data-driven insight sentences, frequent words, and range labels. Dashboard and reports use these helpers instead of separate formulas.

`js/dashboard.js`

`renderDashboard()` gets records, applies selected date/custom/category filters, fills KPIs, and invokes chart/detail/notification renderers. Builds trend, sentiment, rating, product, category, analytics trend, and category-sentiment visualizations; also recent reviews, insights, satisfaction gauge, and keyword cards. Creates/destroys Chart.js instances and uses selected HTML/SVG fallbacks when the chart library is absent.

`js/reviews.js`

Search, filter, sort, and paginate review rows; computes table quick counts; opens review details and response modal; saves replies/status; deletes feedback; exports review CSV; wires review controls. The review detail includes local analysis details and suggested next steps.

`js/insights.js`

Builds additional dataset insight cards: frequent words, highest/lowest product average, complaint topic counts from negative reviews, rating-based satisfaction, and an action recommendation. It derives values from current browser records.

Overview metrics

Total records; mean star rating; positive, neutral and negative counts/shares; and rating-derived satisfaction. The user-selected dashboard date range and category filter are applied before these are calculated.

Analytics view details

Four analysis KPI cards; six-month feedback volume; rating by product; sentiment/category comparisons; category-level volume, average stars, negative count/share; and negative complaint topic counts. Empty results render empty-state rows rather than invented data.

Sentiment scoring idea

score = lexicon polarity + valid rating adjustment; positive if score > 0.35 · negative if score < −0.35 · neutral otherwise

A positive word contributes +1 and a negative word −1; a negation or intensifier among the previous two tokens flips or boosts polarity. Rating adjustment is (rating − 3) × 0.6. Topic and urgent detection are separate word-list matches; they do not prove root cause or emergency status.

> **Interpretation:** The sentiment output is explainable and inexpensive but limited by its lexicon. Sarcasm, context, language variation, domain-specific phrases, and ambiguous wording can yield incorrect classifications.

Forms, operations, Copilot, and reporting

FILE	LOGIC AND OUTPUT	INTERACTION
`js/auth.js`	Switches tabs and password visibility; validates required signup fields and minimum length; checks fixed demo login values; stores demo auth flag and redirects. Password reset explicitly says a real auth service is not configured.	Loads on login page with storage/app helpers.
`js/feedback.js`	Updates star visuals, recommendation controls, character counters and live sentiment. On submit checks required fields and rating; creates an ID/timestamped review with sentiment output; saves it; adds notification; resets form; redirects.	Customer feedback entry and immediate analysis preview.
`js/workspace.js`	Complaint storage/rendering/editing; creates one issue per untracked negative review; chooses priority from urgent flag/rating; persists owner and status. Also parses/imports CSV and first worksheet of Excel, normalizes header aliases, validates rating/text/date, detects duplicates, previews records, and commits valid ones. Implements query matching, calculations, evidence, and message rendering for Copilot.	Owns complaint, upload, and Copilot views.
`js/reports.js`	Aggregates report totals, sentiment counts, insights, and complaint-topic recommendations. Renders summary; exports a CSV with formula-leading fields escaped; creates a PDF using jsPDF; reports when PDF library is unavailable.	Report screen and download controls.
`js/app.js`	Wires app-wide navigation and active states; page title/breadcrumbs; mobile sidebar; theme toggle and persisted selection; profile/notification menus; keyboard search shortcut; modal close; profile editing; demo data reset/clear; notification list/badge; and redirect when dashboard auth flag is absent.	Shared shell behavior across page views.

CSV/Excel import sequence

1. Select a CSV, XLSX, or XLS file (size limit 10 MB).
2. Map supported column-name aliases; read CSV quoted values or first Excel worksheet.
3. Normalize each row and require review text plus integer rating 1–5; validate optional date.
4. Analyze sentiment, mark duplicates by normalized customer/product/text, show preview/status (up to 100 visible rows).
5. Allow commit only for ready, nonduplicate records; cap file at 5,000 records; save and refresh views.

Copilot question sequence

1. User types a question or chooses a sample prompt.
2. `buildCopilotAnswer()` recognizes one of its supported query patterns.
3. It computes a count, average, rank, trend, or topic summary from stored reviews.
4. Where supported, selected review excerpts are shown as evidence.
5. Safe text escaping prevents user/evidence strings being inserted as markup.

Unsupported questions receive a guidance response; this is not open-ended LLM reasoning.

Issue tracking flow: Negative review → duplicate check by linked review ID → title/description and derived priority → owner/status edits → LocalStorage persistence. The issue keeps the source review reference for traceability.

Stylesheets, filters, formulas, and libraries

FILE	STYLING RESPONSIBILITY
`css/style.css`	Base visual layer and earlier/general styles: app layout, typography, controls, cards and legacy selectors that remain in the cascade.
`css/dashboard.css`	Dashboard shell details, notification dropdown/list, section headers, modal-related layouts, and dashboard helpers.
`css/components.css`	Reusable empty states, skeleton placeholders, word cloud and shared component rules.
`css/responsive.css`	Responsive adaptations for compact widths and customer-facing form surfaces.
`css/enterprise.css`	Design tokens and primary theme overrides: light/dark palette, sidebar, topbar, search box, cards, buttons, tables, dialogs, auth layout, breakpoints.
`css/workspace.css`	Newer feature-specific visuals: Analytics cards, Copilot, complaints, import preview/date filters, auth/welcome enhancements, responsive and reduced-motion behavior.

Filtering and computed metrics

Date filter picks records inside today/7/30/90 days or custom bounds. Category selection further narrows overview/analytics metrics. The review table has its own search/filter/sort controls. Copilot answers use the stored dataset directly and do not inherit dashboard filters.

The customer satisfaction value is a scaled mean rating, not a separate survey response:

rating satisfaction (%) = sum of stars ÷ (number of reviews × 5) × 100

CSV and PDF outputs

Review CSV contains the feedback rows; report CSV contains summary metrics and recommendations. Formula-leading CSV values are escaped in report export. PDF report is built from the report summary using jsPDF. It needs the external library to load; print is also available in the interface.

Chart.js renders analytics; SheetJS reads Excel. These vendors are loaded from CDNs by the dashboard page.

External runtime dependencies

LIBRARY/SERVICE	USE	IF IT CANNOT LOAD
Chart.js CDN	Interactive canvas charts	Several charts can use local HTML/SVG fallbacks; CDN access is required for Chart.js behavior.
jsPDF CDN	PDF report download	PDF export reports unavailable; CSV report and print remain alternatives.
SheetJS CDN	Excel workbook parsing	CSV import remains available; Excel import displays an error.
Font Awesome / Google Fonts	Icons and Inter font	Browser fallback fonts/icons may appear; they are visual dependencies.

Keep indicators honest: Dashboard comparisons are descriptive calculations over whichever records are saved and selected. They are not forecasts, causal conclusions, or verified customer population estimates.

How to run and present the project

Local start

From the project directory, run:

```
python3 -m http.server 8000
```

Visit `http://localhost:8000/login.html`. Use the demo account from `README.md`: `admin@feedbackpro.io` / `admin123`.

No npm install or build step is required. Internet access is needed for CDN-based chart, PDF, spreadsheet, font, and icon resources.

Suggested 3-minute demo

1. Sign in with demo account and note the sample dataset.
2. Change date/category filters; compare KPI values and charts.
3. Open a review to explain its sentiment evidence/suggestion.
4. Create complaints from negative reviews and change priority/status.
5. Ask a supported Copilot question and identify the review evidence.
6. Preview an import (optional), then export a CSV/report.

What is implemented and what remains

WORKS IN THIS BROWSER DEMO	NOT PROVIDED BY THIS REPO	NEEDED FOR PRODUCTION SAAS
Dashboard and local analytics; heuristic sentiment/topics; evidence-based supported Copilot queries; feedback form; reviews and replies; local issue tracking; CSV/XLSX preview/import; CSV/PDF summaries; theme/settings; browser persistence.	Server API; durable database; secure user management and password storage; tenant isolation; cloud synchronization; server-side validation/audit; configured LLM; production file storage; centralized access control.	Backend API and database, schema/migrations, identity provider/session security, tenant ownership checks, validation/rate limits, secrets manager, deployment config, backups, observability, privacy policy/retention, and carefully evaluated model service if needed.

Security warning: The authentication flag is stored client-side. Signup is not a real account-creation service, fixed demo credentials are not secure, and browser data is not protected from the local user. Never use real passwords or sensitive customer data in this demo.

Manual verification checklist

- Sign-in and dashboard redirect work; logged-out dashboard redirects to login.
- Sample review counts agree with the visible table and applied filters.
- Submitted review appears in dashboard/review list and notification center.
- Negative review can be tracked once; its priority and status persist after navigation/reload.
- Import preview identifies bad rating, missing text, and duplicates without committing them.
- Supported Copilot answers show values that agree with the dataset/evidence.
- CSV opens with expected columns; PDF export works when jsPDF CDN is reachable.
- Dark/light toggle changes theme and survives reload.

Architecture, files, and core concepts

Use these to rehearse a concise explanation of what the code does and why. Answer cues are reminders of project facts, not scripts to memorize.

01 What problem does FeedbackPro AI solve?
Collects customer feedback, summarizes sentiment and recurring topics, and organizes follow-up.

02 What technology stack does the project use?
Static HTML, CSS, vanilla JavaScript; LocalStorage; selected CDN libraries.

03 Why is there no build command?
Scripts/styles are loaded directly by HTML; no bundler/framework build pipeline.

04 What are the three HTML entry pages, and what is each for?
Dashboard, demo login/signup, customer feedback form.

05 How does the dashboard switch between its sections?
App navigation toggles active page-view sections and updates title/breadcrumb.

06 Why does script load order matter here?
Classic scripts expose global functions; later feature code uses earlier storage/analysis helpers.

07 What is the role of `js/storage.js`?
Sample initialization, LocalStorage access, settings, auth flag, formatting, escaping, toasts.

08 Which LocalStorage keys are important?
Reviews, settings, demo auth, notifications, and complaints.

09 How does a feedback form submission reach the dashboard?
Validation → sentiment analysis → save review → add notification → redirect/rerender.

10 How does `analyzeSentiment()` determine a label?
Lexicon score, negation/intensifier handling, rating adjustment, threshold classification.

11 How is a review topic detected?
Text is matched against topic-specific word lists.

12 How are urgent reviews identified?
Explicit urgent-term matching; it is a heuristic signal for triage.

13 What do `js/analytics.js` helpers calculate?
Date ranges, averages, satisfaction, product ranking, and textual insight summaries.

14 How is customer satisfaction calculated in the app?
Mean star rating divided by five and multiplied by 100.

15 How do dashboard date and category filters affect KPIs?
Records are filtered first; aggregates and charts use that selected subset.

Tip: In a viva, explain the data source and the actual formula before describing what a chart "means." Distinguish observed counts from inferred conclusions.

Features, limitations, and future work

16 What is handled by `js/dashboard.js`?
KPI/chart rendering, analytics detail, recent reviews, insights, and notifications.

17 How does review management work?
Search/filter/sort/paginate, inspect details, save replies/status, delete, export.

18 How does importing handle invalid or duplicate records?
Normalize rows, validate text/rating/date, compare duplicate signatures, preview before commit.

19 How is an issue created from a negative review?
Link to source review, assign suggested priority, then allow owner/status updates.

20 How does the Insights Copilot produce an answer?
Recognized query route → calculation over stored reviews → response and selected evidence.

21 Why is the Copilot not a generative AI model?
No model endpoint or credentials; fixed query branches calculate results locally.

22 Which external libraries are used and why?
Chart.js charts, SheetJS Excel parsing, jsPDF reports; fonts/icons are also external.

23 What happens if the spreadsheet library fails to load?
Excel import shows an unavailable message; CSV can still be imported.

24 What is the difference between average rating and sentiment?
Rating is a numeric customer score; sentiment is a text/rating heuristic class.

25 Why is LocalStorage insufficient for a production SaaS?
Browser-only, per-origin, modifiable, not securely shared, and lacks tenant/server controls.

26 Is the current login secure? Why or why not?
No. Demo flag/credentials are client-side; there is no server auth or password hashing.

27 How can user-submitted text be safely rendered?
Escape HTML before building markup; never trust review or import strings.

28 How would you test the import feature?
Valid CSV/XLSX, quoted comma, empty text, bad rating/date, duplicates, size/row limits.

29 What are limitations of lexicon sentiment analysis?
Context, sarcasm, domain vocabulary, language, and negation can be misread.

30 What would you build next to make this a real SaaS?
API/database, secure identity, tenant isolation, validation, backups, deployment and monitoring.

Follow-up questions to prepare

Be ready to explain what happens when LocalStorage is cleared, how duplicate import signatures are formed, why a recommendation field is distinct from a suggested action, and how the app handles a missing CDN library.

Questions you can ask yourself

Which output is derived from real records? Which parts are heuristics? What evidence supports each insight? What data needs server-side protection? Which test case would expose a false positive?

Data handling, design decisions, and scenarios

Scenario questions help demonstrate understanding beyond memorizing filenames. Explain what happens in the current implementation, then distinguish a production-ready alternative.

31 How does the rating affect sentiment when no sentiment word is present?

The rating adds (rating − 3) × 0.6 to the text score before applying thresholds.

32 How does import duplicate detection work?

Trimmed, lowercased customer name, product, and review text are combined as a signature.

33 Which fields are required for an imported row?

Review text and an integer rating from 1 to 5; date is checked when supplied.

34 Why does the import preview show a limited number of rows?

It displays at most 100 preview rows while counts report the full classified batch.

35 What do CSV quote handling and the parser error protect against?

Commas/newlines inside quoted cells and malformed unterminated quoted fields.

36 Does Insights Copilot apply the current dashboard date/category filters?

No. Its answer branches read the stored review dataset independently.

37 How is the review recommendation answer chosen for imports?

It is derived from rating: 4–5 Yes, 1–2 No, and 3 Maybe.

38 Why are recommendations and sentiment suggestions separate concepts?

Recommendation records customer intent; suggestion is the app's proposed follow-up.

39 How are charts refreshed after data changes?

The dashboard renderers recalculate aggregates and update their chart instances/views.

40 What is an important limitation of the satisfaction percentage?

It is the mean star rating scaled to 100, not a separately measured CSAT survey.

41 How do you explain the distinction between observation and recommendation?

Counts/averages are computed from records; the suggested action is a rule-based prompt.

42 Why seed the application with sample feedback?

It lets a new user explore charts and workflows before collecting or importing data.

43 What happens if LocalStorage contains malformed JSON?

Review/settings accessors use fallback behavior; production storage should validate and report data errors.

44 How can you verify that a Copilot answer is evidence-based?

Compare its count/rating to the matching records and inspect the shown review excerpts.

45 How would you make sentiment more reliable for multilingual reviews?

Use a language-aware evaluated model, preserve evidence, and measure accuracy against labeled data.

Final viva checklist

Know the actual source file for each feature, identify the data store, explain at least one formula, trace one review end to end, and state two clear limitations without describing demo behavior as production security.

Future technical improvement

Introduce automated tests for score boundaries, CSV quoting/duplicates, date filters, and persisted workflows; keep deterministic sample fixtures for repeatable results.